

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом

Студент гр. 4345

Пикалев Н.Г

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

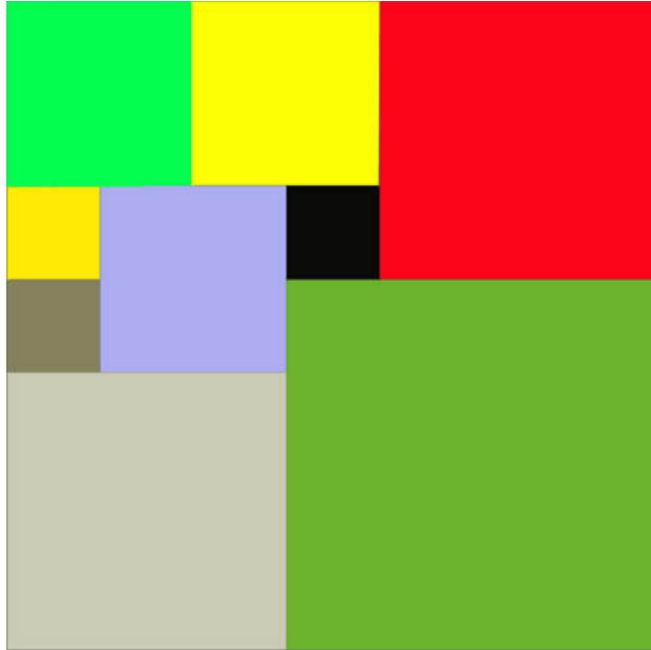
Цель работы.

Изучить и реализовать алгоритм поиска с возвратом.

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу – квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков (квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы – одно целое число N ($2 \leq N \leq 20$).

Выходные данные

Одно число K , задающее минимальное количество обрезков (квадратов), из которых можно построить столешницу (квадрат) заданного размера N . Далее должны идти K строк, каждая из которых должна содержать три целых числа x , y и w , задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка (квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вар. 3р. Рекурсивный бэктрекинг. Исследование кол-ва операций от размера квадрата.

Основные теоретические положения.

Поиск с возвратом (бэктрекинг) – общий метод нахождения решений задачи, в которой требуется полный перебор всех возможных вариантов в некотором множестве M .

Решение задачи методом поиска с возвратом сводится к последовательному расширению частичного решения. Если на очередном шаге такое расширение провести не удастся, то возвращаются к более короткому частичному решению и продолжают поиск дальше. Данный алгоритм позволяет найти все решения поставленной задачи, если они существуют. Для ускорения метода стараются вычисления организовать таким образом, чтобы как можно раньше выявлять заведомо неподходящие варианты. Зачастую это позволяет значительно уменьшить время нахождения решения.

Выполнение работы

Рассмотрим задачу заполнения квадратной столешницы размером $N \times N$ меньшими квадратами со сторонами от 1 до $N-1$. Требуется найти минимальное количество квадратов K , полностью покрывающих область без пустот и перекрытий.

Данную задачу можно разбить на три случая для дальнейшей оптимизации:

1. N – чётное число.

Если N является чётным числом, то количество квадратов, необходимых для заполнения столешницы, будет равняться четырём для всех N . Размер каждого отдельного квадрата равняется $N / 2$

2. N – нечётное составное число.

Если N является нечётным составным числом, то задачу можно привести к заполнению квадрата со стороной, равной наименьшему простому делителю числа N . Возьмём делитель за D , $D \neq 1$. После заполнения необходимо увеличить каждый получившийся маленький квадрат в N / D раз

3. N – нечётное простое число.

Если N является нечётным простым числом, то сперва вставляется квадрат размера $(N + 1) / 2$ в левый верхний угол, а затем два квадрата размера $(N - 1) / 2$ справа и снизу от первого. После вставки первых трёх квадратов остаётся незаполненная область. Данная область заполняется методом рекурсивного бэктрекинга.

Алгоритм рекурсивно находит координаты первой незанятой клетки. В эту позицию алгоритм последовательно пробует установить квадраты всех возможных размеров. Для каждого размера квадрат размещается, после чего рекурсивно вызывается та же функция для заполнения оставшейся области.

Если свободных клеток не нашлось или количество фигур превышает текущий минимум, происходит возврат из рекурсии: последний размещенный квадрат удаляется, и алгоритм пробует следующий возможный размер для предыдущей позиции.

Если для текущей позиции перебраны все возможные размеры, происходит выход из рекурсивного вызова, что эквивалентно извлечению из стека вызовов, и алгоритм продолжает перебор на предыдущем уровне.

Описание функций программы

В программе реализованы следующие функции:

1. Внешняя функция `solve_with_backtracking(N)` – Главная функция, которая организует поиск оптимального разбиения квадрата $N \times N$ на меньшие квадраты.

2. `can_place(x, y, size)` – Проверяет, можно ли разместить квадрат заданного размера в указанной позиции.

3. `place_square(x, y, size, square_id)` – Размещает квадрат в сетке и записывает его в решение.

4. `remove_square(x, y, size)` – Удаляет квадрат из сетки.

5. `find_empty_cell()` – Находит первую пустую клетку для размещения следующего квадрата.

6. `get_max_possible_size(x, y)` – Определяет максимальный размер квадрата, который можно разместить в данной позиции.

7. `backtrack(squares)` – Основная рекурсивная функция поиска с возвратом.

Исходный код программы см. в приложении А.

Результаты тестирования программы см. в таблице 1.

Таблица 1 – Тестирование программы

Размер столешницы	Результат
7	9 1 1 4 1 5 3 5 1 3 5 4 2 7 4 1 4 5 1 7 5 1 4 6 2 6 6 2

9	6 1 1 6 1 7 3 7 1 3 7 4 3 4 7 3 7 7 3
16	4 1 1 8 9 1 8 1 9 8 9 9 8
19	13 1 1 10 1 11 9 11 1 9 11 10 3 14 10 6 10 11 1 10 12 1 10 13 4 14 16 1 15 16 1 16 16 4 10 17 3 13 17 3

Было проведено исследование зависимости количества операций от размера столешницы N. Результаты исследования см. на Рис. 1

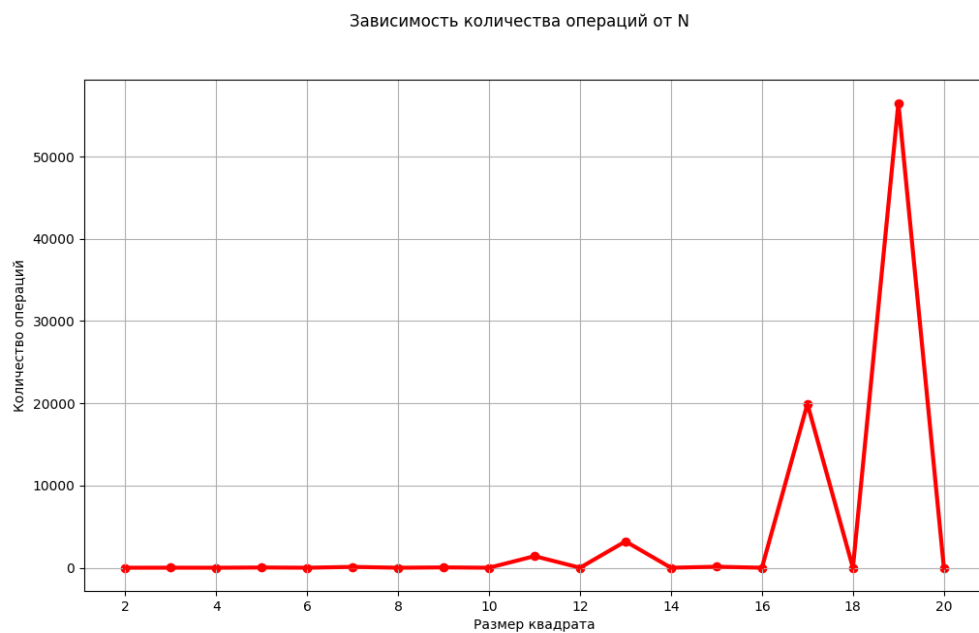


Рисунок 1 – Зависимость количества операций от N

Из графика можно понять, что меньше всего операций выполняется при чётных значениях N , для всех значений ответ равняется четырём. Благодаря использованию масштабирования удалось сократить количество операций в программе для нечётных составных значений N , оно лишь немного больше, чем для чётных. Для нечётных простых чисел количество операций является наибольшим.

Выводы.

В ходе выполнения данной лабораторной работы был реализован алгоритм поиска с возвратом, позволяющий найти минимальное количество квадратов для покрытия области квадратной столешницы со стороной N .

Реализация выполнена с использованием рекурсивного подхода. В программу добавлены функции проверки чётности входного числа, поиск его делителей, что ускорило выполнение программы для чётных и составных чисел.

Исследование зависимости количества операций от размера столешницы N показало, что для простых чисел наблюдается резкий рост количества операций, что потребовало внедрения механизма отсечений изначально неверных комбинаций квадратов для нахождения решения за приемлемое время.

Было проведено тестирование программы при разных входных данных, программа успешно выполняется и выводит результат своей работы на экран.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb1.py

```
def solve_with_backtracking(N):
    grid = [[0] * N for _ in range(N)]

    best_solution = None
    operations_counter = [0]
    best_count = float('inf')

    def can_place(x, y, size):
        if x + size > N or y + size > N:
            return False

        for i in range(size):
            for j in range(size):
                if grid[y + i][x + j] != 0:
                    return False
        return True

    def place_square(x, y, size, square_id):
        print(f"Размещаем в клетке {x, y} квадрат размером
{size}")
        for i in range(size):
            for j in range(size):
                grid[y + i][x + j] = square_id
            operations_counter[0] += 1

    def remove_square(x, y, size):
        print(f"Удаляем из клетки {x, y} квадрат размером
{size}")
        for i in range(size):
            for j in range(size):
                grid[y + i][x + j] = 0
            operations_counter[0] += 1

    def find_empty_cell():
        for y in range(N):
            for x in range(N):
                if grid[y][x] == 0:
                    return(x, y)
        return None

    def get_max_possible_size(x, y):
        if grid[y][x] != 0:
            return 0

        max_size = 1
        while (x + max_size <= N and y + max_size <= N):
            can_expand = True

            for i in range(max_size):
                if grid[y + max_size - 1][x + i] != 0:
                    can_expand = False
                    break
```

```

        if grid[y + i][x + max_size - 1] != 0:
            can_expand = False
            break

    if not can_expand:
        break

    max_size += 1

    return max_size - 1

def backtrack(squares):
    nonlocal best_solution, best_count

    if N % 2 == 0:
        if can_place(0, 0, N//2):
            size = N//2

            place_square(0, 0, size, 1)
            squares.append((1, 1, size))
            print(f"Размещаем в клетке {1, 1} квадрат
размером {size}")

            place_square(N//2, 0, size, 2)
            squares.append((N//2+1, 1, size))
            print(f"Размещаем в клетке {N//2+1, 1} квадрат
размером {size}")

            place_square(0, N//2, size, 3)
            squares.append((1, N//2+1, size))
            print(f"Размещаем в клетке {1, N//2+1} квадрат
размером {size}")

            place_square(N//2, N//2, size, 4)
            squares.append((N//2+1, N//2+1, size))
            print(f"Размещаем в клетке {N//2+1, N//2+1}
квадрат размером {size}")

        if N % 2 == 1 and N > 1 and N != 9 and N != 15:
            if can_place(0, 0, N//2 + 1):
                big_size = N//2 + 1
                small_size = N//2

                place_square(0, 0, big_size, 1)
                squares.append((1, 1, big_size))
                print(f"Размещаем в клетке {1, 1} квадрат
размером {big_size}")

                place_square(0, big_size, small_size, 2)
                squares.append((1, big_size+1, small_size))
                print(f"Размещаем в клетке {1, big_size+1}
квадрат размером {big_size}")

                place_square(big_size, 0, small_size, 3)
                squares.append((big_size+1, 1, small_size))
                print(f"Размещаем в клетке {big_size+1, 1}
квадрат размером {big_size}")

```

```

    if N == 9 or N == 15:
        if can_place(0, 0, N//2 + 1):
            small_size = N//3
            big_size = small_size * 2

            place_square(0, 0, big_size, 1)
            squares.append((1, 1, big_size))
            print(f"Размещаем в клетке {1, 1} квадрат
размером {big_size}")

            place_square(0, big_size, small_size, 2)
            squares.append((1, big_size+1, small_size))
            print(f"Размещаем в клетке {1, big_size+1}
квадрат размером {big_size}")

            place_square(big_size, 0, small_size, 3)
            squares.append((big_size+1, 1, small_size))
            print(f"Размещаем в клетке {big_size+1, 1}
квадрат размером {big_size}")

        empty = find_empty_cell()

        if empty is None:
            if len(squares) < best_count:
                best_count = len(squares)
                best_solution = squares.copy()
            return

        x, y = empty

        if len(squares) >= best_count:
            return

        max_size = get_max_possible_size(x, y)
        max_size = min(max_size, N - 1)

        print(f"Количество элементов: {len(squares)}")

        for size in range(max_size, 0, -1):
            if can_place(x, y, size):
                square_id = len(squares) + 1
                place_square(x, y, size, square_id)
                squares.append((x + 1, y + 1, size))

                backtrack(squares)

                squares.pop()
                remove_square(x, y, size)

        backtrack([])
        return best_solution, operations_counter[0]

if __name__ == "__main__":
    N = int(input())

    squares, count_operations = solve_with_backtracking(N)

    if squares:

```

```
    print(len(squares))
    for x, y, size in squares:
        print(x, y, size)
    print(f"Количество операций: {count_operations}")
else:
    print("Решение не найдено")
```